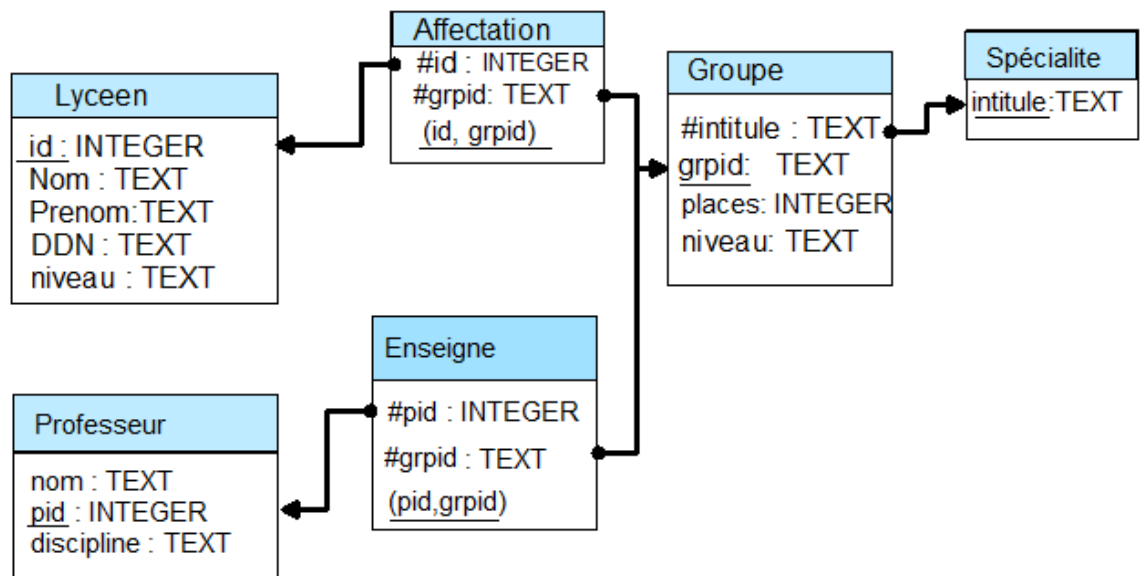


TD Recherche de données dans une relation (table)

Le schéma de la base de données utilisée est le suivant:



Exercices :

1.

Afficher les valeurs des attributs `grpId` et `intitule` des entités appartenant à la relation Groupe.

Entrée[]: **SELECT** grpId,intitule **FROM** Groupe;

2.

D'après les informations obtenues et le schéma de la base de données, serait-il possible d'ajouter dans la relation Groupe chacune des entités suivantes:

- ("Sciences de l'Ingénieur","1SIG1",24,"Prem")
- ("Sciences de l'Ingénieur","1SIG3",24,"Prem")
- ("Sciences de l'Ingénieur","TSVTG3",24,"Term")
- ("Sciences de l'Ingénieur","TSIG2",24)

Quand ce n'est pas possible, expliquer pourquoi.

- La première entrée n'est pas possible car `grp_id` est une clé primaire et une valeur 1SIG1 existe déjà dans la table.
- La deuxième et la troisième entrée respectent le schéma de la relation Groupe et les contraintes : la clé étrangère "Sciences de l'Ingénieur" existe puisque cette valeur figure dans la table, et les identifiants "1SIG3" et "TSVTG3" ne sont pas déjà utilisés.
- La dernière entrée ne respecte pas le schéma de la relation Groupe et sera donc rejetée.

On se doute bien que valider la troisième entrée n'est en fait pas souhaitable car l'observateur humain a bien décelé que "TSVTG3" n'est sans doute pas un identifiant approprié pour un groupe de Sciences de l'Ingénieur, mais le schéma de la base ne suffit pas à nous indiquer s'il existe une contrainte supplémentaire à respecter à ce niveau.

3.

a. Afficher toutes les valeurs de l'attribut `intitule` de la table Groupe.

```
Entrée[ ]: SELECT intitule FROM Groupe;
```

b. Modifier ensuite la commande pour supprimer les doublons parmi les réponses.

```
Entrée[ ]: SELECT DISTINCT intitule FROM Groupe;
```

4.

Parmi les affirmations suivantes, concernant l'attribut `intitule` de la relation *Groupe*, qu'on notera `Groupe.intitule`, et celui de la relation *Specialite*, qu'on notera `Specialite.intitulé`, quelles sont celles qui sont exactes (expliquer pourquoi) :

- La liste des valeurs de `Groupe.intitule` est forcément la même que la liste des valeurs de `Specialite.intitule`
- `Groupe.intitule` peut prendre des valeurs ne figurant pas dans la liste des valeurs de `Specialite.intitule`
- `Specialite.intitule` peut prendre des valeurs ne figurant pas dans la liste des valeurs de `Groupe.intitule`
- Les valeurs prises par `Groupe.intitule` sont forcément toutes différentes
- Les valeurs prises par `Specialite.intitule` sont forcément toutes différentes

Réponses:

- faux : voir 3ème réponse ci-dessous
- faux : `Groupe.intitule` est une clé étrangère faisant référence à `Specialite.intitule`
- vrai : Il n'y a pas d'obligation à ce que toutes les valeurs possibles pour la clé étrangère soient effectivement utilisées
- faux : `Groupe.intitule` n'est pas une clé primaire de *Groupe*, une même valeur peut donc apparaître plusieurs fois dans la table pour cet attribut
- vrai : `Specialite.intitule` est une clé primaire de la table *Specialite*

5. Tris

a. Afficher le niveau, l'intitulé et le `grpid` des groupes par ordre alphabétique de leur intitulé.

Entrée[] : **SELECT** niveau,intitule,grpid **FROM** Groupe **ORDER BY** intitule;

b. Afficher le niveau, l'intitulé et le `grpid` des groupes par ordre alphabétique d'intitulé puis de niveau.

Entrée[]: **SELECT** niveau,intitule,grpId **FROM** Groupe **ORDER BY** intitule, niveau;

c. Quel est l'effet de chacun des mots-clé DESC et ASC, écrit juste après une clé de tri ?

DESC donne un tri par ordre décroissant, ASC un tri par ordre croissant (option par défaut)

d. Afficher les identifiants grpId et les intitulés des groupes de spécialité de 1ère par ordre de nombre de places décroissant.

Entrée[]: **SELECT** grpId,intitule **FROM** groupe **ORDER BY** places **DESC**;

6. Filtres

a. Afficher les identifiants grpId des groupes de spécialité ayant au moins 30 places.

Entrée[]: **SELECT** grpId **FROM** Groupe **WHERE** places>=30;

b. Afficher les intitulés des spécialités dont les groupes ont au moins 30 places.

Entrée[]: **SELECT** intitule **FROM** Groupe **WHERE** places>=30;

c. Afficher les intitulés des spécialités dont les groupes ont au moins 30 places, en supprimant les doublons (voir exercice 3).

Entrée[]: **SELECT DISTINCT** intitule **FROM** Groupe **WHERE** places>=30;

d. Afficher les valeurs de l'attribut 'intitule' de la tables Groupe, contenant le mot Science, en supprimant les doublons.

Entrée[]: **SELECT DISTINCT** intitule **FROM** Groupe **WHERE** intitule **LIKE** "%Sciences%";

7. Filtres et opérateurs logiques

a. Afficher les identifiants `grpid` des groupes de spécialité Physique-Chimie.

Entrée[]: **SELECT** `grpid` **FROM** Groupe **WHERE** `intitule="Physique-Chimie"`;

b. Afficher les identifiants `grpid` des groupes de spécialité Physique-Chimie de niveau Première.

Entrée[]: **SELECT** `grpid` **FROM** Groupe
WHERE `intitule="Physique-Chimie"` **AND** `niveau="Prem"`;

c. Afficher les identifiants `grpid` des groupes de spécialité Physique-Chimie et Sciences de la Vie et de la Terre.

Entrée[]: **SELECT** `grpid` **FROM** Groupe
WHERE `intitule="Physique-Chimie"`
OR `intitule="Sciences de la Vie et de la Terre"`;

d. Afficher les identifiants `grpid` des groupes de spécialité Physique-Chimie et Sciences de la Vie et de la Terre de niveau Première.

Entrée[]: **SELECT** `grpid` **FROM** Groupe
WHERE (`intitule="Physique-Chimie"`
 OR `intitule="Sciences de la Vie et de la Terre"`)
 AND `niveau = "Prem"`;

8. Agrégateurs

a. Afficher le total des places de spécialités disponibles, selon la table Groupe.

Entrée[]: **SELECT** `SUM(places)` **FROM** Groupe;

b. Afficher la taille moyenne d'un groupe de spécialité, selon la table Groupe.

```
Entrée[ ]: SELECT AVG(places) FROM Groupe;
```

Si on veut un nombre entier comme résultat on peut utiliser les fonctions `round` (arrondi à l'entier le plus proche) ou `floor` (arrondi à l'entier inférieur)

```
Entrée[ ]: SELECT round(AVG(places)) FROM Groupe;
```

```
Entrée[ ]: SELECT floor(AVG(places)) FROM Groupe;
```

9. Agrégateurs et filtres

a. Afficher le nombre de groupes ayant plus de 30 places.

```
Entrée[ ]: SELECT COUNT(*) FROM Groupe WHERE places>30;
```

```
Entrée[ ]: SELECT COUNT(grpid) FROM Groupe WHERE places>30;
```

b. Afficher le total des places disponibles en Physique-Chimie en Terminale.

```
Entrée[ ]: SELECT SUM(places)
FROM Groupe
WHERE intitule="Physique-Chimie"
AND niveau="Term";
```

c. Afficher la plus grande valeur de l'attribut `id` dans la table `Lyceen`, puis la plus petite.

```
Entrée[ ]: SELECT MAX(id) FROM Lyceen;
```

```
Entrée[ ]: SELECT MIN(id) FROM Lyceen;
```

d. Afficher la plus grande valeur de l'attribut `id` pour un lycéen de Première dans la table `Lyceen`.

```
Entrée[ ]: SELECT MAX(id) FROM Lyceen WHERE niveau = "Prem";
```

e. Afficher le nom du lycéen de Seconde ayant le plus petit identifiant, en utilisant l'agrégateur *MIN*.

```
Entrée[ ]: SELECT nom,MIN(id)
           FROM Lyceen
           WHERE niveau="Sec";
```

Moyennant quelques efforts il est possible de n'afficher QUE le nom sans l'identifiant, puisque seul le nom était demandé :

Les données extraites par SELECT peuvent être utilisées comme une table dans une requête:

```
Entrée[ ]: SELECT nom FROM
           (SELECT nom,MIN(id)
            FROM Lyceen
            WHERE niveau="Sec");
```

Une donnée extraite par SELECT peut aussi être utilisée comme une valeur dans une requête:

```
Entrée[ ]: SELECT nom
           FROM Lyceen
           WHERE id=(
               SELECT MIN(id)
               FROM Lyceen
               WHERE niveau="Sec" );
```

Sans utiliser l'agrégateur *MIN* on peut obtenir le même résultat avec LIMIT

```
Entrée[ ]: SELECT nom
           FROM Lyceen
           WHERE niveau="Sec"
           ORDER BY id
           LIMIT 1
```

```
Entrée[ ]:
```

